

Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title:

Playful AI: Intelligent Board Game Opponents and Advisors

Team Name:

Team Masterminds

Team Members:

- Saiveer Pendkar
 - Saicharan Punna
 - Kishan Pathlavath
 - Aravind Perala
-

Phase-1: Brainstorming & Ideation

Objective:

- Playful AI: Intelligent Board Game Opponents and Advisors
- It is the type of project where humans can be play the game with ai

Key Points:

1. **Problem Statement:** Creativity and content generation
 2. **Proposed Solution:** We created the game based on playful ai where human play game with ai
 3. **Target Users:** Mostly all are benefit from this project because its completely based on game
 4. **Expected Outcome:** success fully we created a playful ai game i.e, connect 4 game it's a mind-based game in which we connect the color from diagonally, horizontally, and vertically.
-

Phase-2: Requirement Analysis

Objective:

- Technical and functional Requirements

Key Points:

1. **Technical Requirements:** Python (any programming language), game board implementation, game logic, AI opponent.
 2. **Functional Requirements:** Game rules and logic, User interaction, game UI.
 3. **Constraints & Challenges:** Difficulty balancing, unpredictable behavior of AI.
-

Phase-3: Project Design

Objective:

- Create the architecture and user flow.

Key Points:

1. **System Architecture Diagram:** (Simple sketch or flowchart)
 2. **User Flow:** (How a user will interact with the project)
 3. **UI/UX Considerations:** (If applicable, wireframe or basic layout)
-

Phase-4: Project Planning (Agile Methodologies)

Objective:

-  **Architecture Overview**
- **1. Frontend (UI/UX)**
- **Tech Stack:** React.js (for web), Unity (for 3D), or Flutter (for mobile).
- **UI Components:**
- Game board (7x6 grid)
- Player vs AI mode selection
- Move history display
- Restart & Undo buttons

- AI difficulty selection (Easy, Medium, Hard)
- **2. Backend (Game Logic & AI)**
- **Tech Stack:** Python (Flask/Django), Node.js (Express)
- **Key Components:**
- **Game Engine:** Manages board state, move validation, win detection
- **AI Algorithm:** Uses **Minimax with Alpha-Beta Pruning** or **Monte Carlo Tree Search (MCTS)**
- **Move Evaluator:** Assigns scores to different board states for the AI
- **Multiplayer Mode (Optional):** WebSocket-based real-time gameplay
- **3. Database (Optional, for Leaderboards & Game History)**
- **Tech Stack:** Firebase, PostgreSQL, MongoDB
- **Data Stored:**
- Player profiles & scores
- AI difficulty settings & game history
- **4. Deployment**
- **Cloud Hosting:** AWS, Firebase, or Heroku
- **Real-time Communication (if Multiplayer):** WebSockets or Firebase Realtime Database
-  **User Flow Statements**
- **1. Game Start**
- User opens the game and selects "Player vs AI" or "Player vs Player".
- If AI is chosen, the user selects difficulty (Easy, Medium, Hard).
- The game board initializes with an empty 7x6 grid.
- **2. Player Move**
- The player clicks/taps on a column to drop a piece.
- The game engine validates and updates the board state.
- **3. AI Move (if enabled)**

- AI evaluates the board using **Minimax/MCTS**.
- AI selects the best possible move and updates the board.
- **4. Win/Loss/Draw Check**
- After each move, the game engine checks for:
 - **Win:** Four pieces in a row (horizontal, vertical, diagonal).
 - **Draw:** No more available moves.
- **5. End of Game**
- If the game is over, a message displays the winner or a draw notification.
- User can restart the game or view the move history.

Key Points:

1. Sprint Planning:



Sprint 1: Core Game Mechanics

- **We will develop** the **game board UI and basic interactions** so that players can drop pieces into columns.
 - **We will achieve** a **fully functional two-player mode** with **win/loss/draw detection** to complete the core gameplay.
-



Sprint 2: AI Implementation

- **We will implement** the **AI opponent using Minimax with Alpha-Beta Pruning** so that the AI can make strategic moves.
 - **We will achieve** different **difficulty levels (Easy, Medium, Hard)** to offer a varied gameplay experience.
-



Sprint 3: UI Enhancements & Multiplayer

- **We will enhance** the **UI with animations and a better game board design** so that players experience smooth and intuitive gameplay.
 - **We will implement** **real-time multiplayer mode using WebSockets/Firebase** to allow players to compete online.
-



Sprint 4: Final Features & Deployment

- **We will refine the game settings, leaderboards, and move history tracking** so that players can customize their experience and track their performance.
- **We will achieve a fully optimized and deployed game** so that users can play seamlessly on web or mobile.

2. Task Allocation:

UI developer: - saiveer who develops the interface

Frontend developer: - saicharan he develops the api in the program

Support developers: - Kishan and Aravind both are helps in the code and development of the project

3. Timeline & Milestones: for all this project we taken approximately 15+ hours

Phase-5: Project Development

Objective:

- Using python, we create the connect 4 game

Key Points:

1. **Technology Stack Used:** Python programming language, Vertex ai etc..
 2. **Development Process:** analyzing and algorithm, creating ui and backend development implementation of API
 3. **Challenges & Fixes:** Obstacles faced at the implementation of api and this can be solved by suggestion by mentor
-

Final Submission

1. **Demo Video (3-5 Minutes)**
 2. **GitHub/Code Repository Link**
-